

# System Architecture Document (SAD)

## IntelliPlant AI

### AI-Powered Industrial Knowledge Intelligence Platform

**Version:** 1.0

**Document Type:** System Architecture Document (SAD)

**Prepared For:** ET AI Hackathon 2026

**Prepared By:** Team IntelliPlant AI

---

## 1. Purpose

The purpose of this System Architecture Document (SAD) is to define the overall architecture, components, interactions, data flow, technologies, and deployment strategy for IntelliPlant AI.

The system converts fragmented industrial documents into a unified knowledge intelligence platform using:

- OCR
  - Hybrid RAG
  - Knowledge Graphs
  - Multi-Agent Systems
  - Vector Search
  - LLMs
- 

## 2. Architecture Goals

The architecture should provide:

### Scalability

Support future growth.

---

### Modularity

Independent services.

---

## Explainability

Source citations and graph evidence.

---

## Reliability

Fault tolerant services.

---

## Maintainability

Microservice-based design.

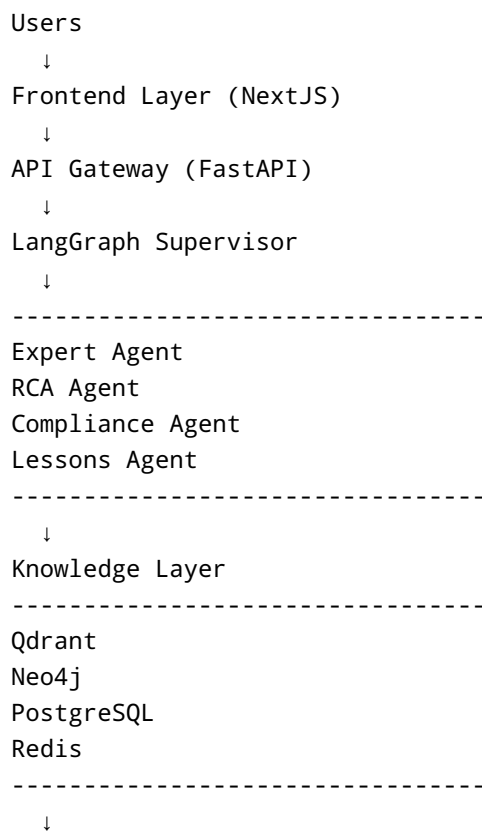
---

## Extensibility

Future IoT and SCADA integrations.

---

# 3. High-Level Architecture



Document Intelligence Layer

-----  
OCR  
Chunking  
Embedding  
Entity Extraction  
-----

↓

Storage Layer

-----  
S3 / MinIO  
-----

↓

LLM Layer  
Gemini / OpenAI

---

## 4. Architecture Principles

### Service-Oriented

Independent modules.

---

### Stateless APIs

REST-based services.

---

### Event Driven

Background processing.

---

### Explainable AI

Citation-based responses.

---

### Graph-Centric Intelligence

Knowledge Graph + Vector Search.

---

## 5. System Components

### Frontend Layer

Technology:

NextJS

Responsibilities:

- Chat UI
  - Dashboard
  - Graph Visualization
  - Document Management
- 

### API Gateway

Technology:

FastAPI

Responsibilities:

- Authentication
  - Request routing
  - Rate limiting
  - Validation
- 

### AI Orchestrator

Technology:

LangGraph

Responsibilities:

- Agent coordination
  - Task routing
  - Context management
-

## Retrieval Layer

Responsibilities:

Hybrid Search

Combines:

- Vector Search
- BM25 Search
- Graph Search

---

## Storage Layer

Stores:

- Documents
- Metadata
- Embeddings
- Graph relationships

---

## 6. Microservice Architecture

```
Frontend Service
  ↓
API Gateway Service
  ↓
-----
Auth Service
User Service
Document Service
Search Service
Agent Service
Graph Service
Audit Service
-----
  ↓
-----
PostgreSQL
Neo4j
Qdrant
Redis
```

MinIO

-----

## 7. AI Architecture

User Query



Supervisor Agent



-----

Expert Copilot Agent

RCA Agent

Compliance Agent

Lessons Agent



Hybrid Retrieval



LLM



Response with Citations

## 8. Multi-Agent Architecture

### Supervisor Agent

Responsible for:

- Task routing
- Context management

### Expert Copilot Agent

Handles:

- Question answering
- SOP retrieval

## RCA Agent

Handles:

- Failure analysis
  - Similar incident discovery
- 

## Compliance Agent

Handles:

- OISD
  - ISO
  - Factory Act
- 

## Lessons Agent

Handles:

- Incident patterns
  - Near misses
- 

# 9. Document Intelligence Architecture

```
graph TD; A[Upload Document] --> B[OCR]; B --> C[Metadata Extraction]; C --> D[Entity Extraction]; D --> E[Chunking]; E --> F[Embedding Generation]; F --> G[Qdrant]; G --> H[Knowledge Graph Builder];
```

↓  
Neo4j

---

## 10. OCR Architecture

Tools:

Docling

PaddleOCR

Tesseract

Pipeline:

```
graph TD; A[Image/PDF] --> B[OCR]; B --> C[Layout Detection]; C --> D[Table Extraction]; D --> E[Text Normalization]; E --> F[Structured JSON];
```

---

## 11. Hybrid Search Architecture

**Vector Search**

Qdrant

Purpose:

Semantic retrieval

---



## BM25 Search

Keyword matching

---

## Graph Search

Neo4j

Relationship traversal

---

## Context Fusion

Merge all results.

---

User Query

↓

Query Rewriter

↓

-----  
Qdrant Search

BM25 Search

Neo4j Search  
-----

↓

Context Fusion

↓

LLM

---

# 12. Knowledge Graph Architecture

## Nodes

Equipment

Manual

Incident

Operator

Failure

Inspection

Regulation

Work Order

---

## Relationships

CONNECTED\_TO

FAILED\_IN

CAUSES

DESCRIBED\_BY

REQUIRES

SIMILAR\_TO

REFERENCES

---

# 13. Database Architecture

## PostgreSQL

Stores:

Users

Documents

Metadata

Audit logs

Sessions

---

## Neo4j

Stores:

Knowledge graph

Relationships

---

## Qdrant

Stores:

Embeddings

Semantic chunks

---

## Redis

Stores:

Cache

Session state

Agent memory

---

## MinIO

Stores:

Original documents

PDFs

Images

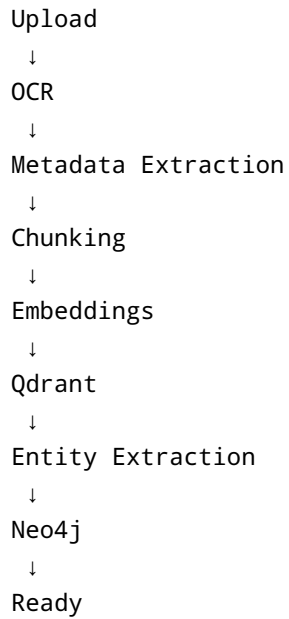
Files

---

# 14. Data Flow Architecture

## Document Ingestion Flow

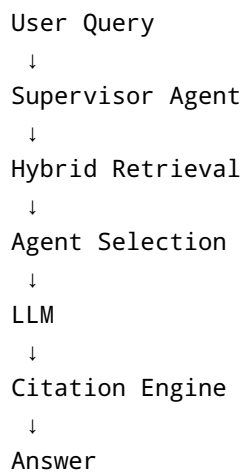
```
graph TD; Upload --> OCR; OCR --> MetadataExtraction[Metadata Extraction]; MetadataExtraction --> Chunking; Chunking --> Embeddings; Embeddings --> Qdrant; Qdrant --> EntityExtraction[Entity Extraction]; EntityExtraction --> Neo4j; Neo4j --> Ready;
```



---

## Question Answering Flow

```
graph TD; UserQuery[User Query] --> SupervisorAgent[Supervisor Agent]; SupervisorAgent --> HybridRetrieval[Hybrid Retrieval]; HybridRetrieval --> AgentSelection[Agent Selection]; AgentSelection --> LLM; LLM --> CitationEngine[Citation Engine]; CitationEngine --> Answer;
```



---

# 15. Security Architecture

Authentication

JWT

---

Authorization

RBAC

Roles:

Admin

Manager

Engineer

Technician

---

Encryption

HTTPS

TLS

---

Password Storage

bcrypt

---

Input Validation

Pydantic

---

Audit Logging

Enabled

---

Rate Limiting

Enabled

---

## 16. Deployment Architecture

Docker Containers

Frontend Container

Backend Container

Worker Container

Redis Container

Qdrant Container

Neo4j Container

PostgreSQL Container

MinIO Container

---

## 17. Background Processing

Technology:

Celery + Redis

Processes:

OCR

Embedding

Entity extraction

Knowledge graph updates

Notifications

Audit logging

---

## 18. Logging Architecture

Application Logs

API Logs

Agent Logs

LLM Logs

Audit Logs

---

Tools:

ELK Stack

Prometheus

Grafana

---

## 19. Fault Tolerance

Retry mechanism

Fallback retrieval

Circuit breaker

Dead letter queue

Caching

Graceful degradation

---

## 20. Scalability

Horizontal scaling

Containerized services

Distributed vector search

Load balancing

Stateless APIs

---

## 21. Performance Targets

| Metric            | Target  |
|-------------------|---------|
| Chat Response     | <5 sec  |
| Search Latency    | <2 sec  |
| OCR Time          | <20 sec |
| Upload Processing | <30 sec |
| Uptime            | 99%     |
| Concurrent Users  | 1000+   |

---

## 22. Monitoring

Prometheus

Grafana

Health checks

Error tracking

Latency monitoring

Token usage monitoring

---

## 23. Future Architecture Extensions

IoT Integration

SCADA Integration



Digital Twins

SAP Integration

Predictive Maintenance

Real-Time Sensor Intelligence

Voice Copilot

Mobile Application

Edge Deployment

---

## 24. Technology Stack

### Frontend

NextJS

TailwindCSS

Shadcn

TypeScript

---

### Backend

FastAPI

LangGraph

Celery

Redis

---

### Databases

PostgreSQL

Neo4j

Qdrant

Redis

MinIO

---

## **AI**

Gemini

OpenAI

Sentence Transformers

BGE Embeddings

---

## **OCR**

Docling

PaddleOCR

Tesseract

---

## **Monitoring**

Prometheus

Grafana

ELK

---

## **Deployment**

Docker

Kubernetes

AWS

Azure

## 25. Conclusion

IntelliPlant AI follows a modular, scalable, and explainable architecture built around GraphRAG and Multi-Agent AI. The architecture enables industrial organizations to transform fragmented documents into an intelligent operational brain capable of delivering knowledge, insights, compliance support, and failure intelligence.